

UNAM

Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelado y Programación

Práctica 03

Equipo Polimórfo

Juárez Larrondo Omar Alejandro
(322245244)

Soto Mendoza Ismael de Jesús
(322210422)

Vega Navas Saúl (322088267)

Identificación de la Práctica

Este documento corresponde a la **Práctica 03** de la materia *Modelado y Programación* de la Facultad de Ciencias, UNAM. El objetivo principal es implementar adecuadamente los patrones de diseño **Iterator**, **Builder** y **Factory** en la resolución del problema propuesto para la Academia Ninja de la Aldea de las Ciencias.

1.1. Descripción del Problema

Se implementó un sistema integral de gestión para la Academia Ninja de la Aldea de las Ciencias, donde se coordinan actividades de entrenamiento para inspirar a las nuevas generaciones a convertirse en ninjas. El sistema gestiona la información de aspirantes interesados almacenados en una estructura HashTable, que incluye nombre, edad, clan de procedencia entre cinco clanes disponibles (Fuchiha, Osomaki, Naca, Mortalika, Akipichi) y un nivel de habilidad entre uno y tres, representando las capacidades básicas de combate ninja.

La Academia cuenta con ninjas voluntarios organizados en una estructura Array que almacena información detallada de cada instructor, incluyendo nombre, edad, clan de procedencia, rango ninja clasificado entre genin, chunin y jonin, y nivel de habilidad avanzado entre cuatro y seis puntos. La formación de grupos operativos sigue reglas estrictas donde cada ninja voluntario actúa como líder de grupo con capacidad limitada según su rango, asignando un aspirante máximo para genin, hasta dos aspirantes para chunin, y hasta tres aspirantes para jonin.

El sistema integra un subsistema de gestión de paquetes de herramientas ninja que incluye cinco tipos fundamentales de equipamiento con pesos específicos establecidos para cada herramienta. Las herramientas disponibles comprenden kunais, shurikens, papeles bomba, bombas de humo y botiquines, organizados en tres paquetes prefabricados que satisfacen diferentes necesidades operativas. El paquete básico contiene un kunai, un shuriken y un botiquín para entrenamientos elementales, mientras que el paquete avanzado incluye dos shurikens, tres papeles bomba, dos bombas de humo y dos botiquines para operaciones de nivel intermedio. El paquete táctico incorpora tres kunais, dos shurikens, cuatro papeles bomba y dos bombas de humo para misiones especializadas de alto nivel.

La asignación de campos de entrenamiento se determina automáticamente según la suma total de los niveles de habilidad de todos los integrantes del grupo, incluyendo el líder voluntario y todos los aspirantes asignados. Los grupos con suma menor o igual a siete puntos entrenan en el Valle del Dragón, caracterizado por su terreno básico ideal para fundamentos ninja, aquellos con suma entre ocho y once puntos utilizan el Bosque Sombrío con obstáculos naturales de dificultad intermedia, mientras que grupos con suma mayor o igual a doce puntos acceden a la Montaña Espiritual, reservada para entrenamientos avanzados con desafíos extremos.

Análisis de Implementación de Patrones

2.1. Patrón Iterator

¿Por qué se eligió esta implementación?

El patrón Iterator fue implementado para proporcionar un mecanismo uniforme de acceso secuencial a las colecciones heterogéneas de aspirantes y voluntarios, eliminando la necesidad de exponer las estructuras de datos internas y permitiendo que el código cliente navegue por ambas colecciones utilizando una interfaz consistente. Esta elección se fundamenta en la necesidad de abstraer las diferencias entre la HashTable utilizada para almacenar aspirantes y el Array empleado para gestionar voluntarios, proporcionando un punto de acceso común que facilita operaciones de iteración sin comprometer el encapsulamiento de datos.

La implementación se materializa a través de la interfaz genérica `Iterator<T>` que define el contrato fundamental con métodos `hasNext()` para verificar disponibilidad de elementos adicionales y `next()` para obtener el siguiente elemento en la secuencia de iteración. Esta interfaz se especializa en dos implementa-

ciones concretas que manejan las particularidades de cada estructura de datos subyacente, manteniendo la uniformidad en el acceso mientras respetando las características específicas de almacenamiento.

La clase `EstudianteNinjaIterator` implementa la navegación específica para la `HashTable` de aspirantes, gestionando internamente la conversión desde la estructura hash hacia una secuencia ordenada que permite acceso secuencial predecible. Esta implementación encapsula la complejidad inherente de iterar sobre estructuras hash, proporcionando un mecanismo transparente que permite al código cliente procesar todos los aspirantes registrados sin conocimiento de la implementación subyacente.

Paralelamente, `VoluntarioIterator` maneja la iteración sobre el `Array` de ninjas voluntarios, aprovechando la naturaleza secuencial natural de la estructura para proporcionar acceso eficiente y directo a cada elemento. Esta implementación optimiza el rendimiento de acceso mientras mantiene la consistencia de interfaz con el iterador de aspirantes, asegurando que el código cliente pueda procesar ambas colecciones utilizando patrones de código idénticos.

La arquitectura de iterator permite que la clase `Academia` implemente la lógica de formación de grupos sin acoplarse a las estructuras de datos específicas, utilizando los iteradores para acceder secuencialmente a voluntarios disponibles y aspirantes sin asignación. Esta separación de responsabilidades facilita modificaciones futuras en las estructuras de almacenamiento sin requerir cambios en la lógica de negocio, demostrando la versatilidad y mantenibilidad que proporciona el patrón `Iterator` en sistemas complejos de gestión de datos.

2.2. Patrón Builder

¿Por qué se eligió esta implementación?

El patrón `Builder` fue implementado para gestionar la construcción compleja de paquetes de herramientas ninja, proporcionando un mecanismo flexible para crear tanto paquetes predefinidos como configuraciones personalizadas según los requerimientos específicos de cada grupo de entrenamiento. Esta elección se fundamenta en la necesidad de manejar la complejidad inherente de combinar múltiples tipos de herramientas en cantidades variables, evitando la proliferación de constructores con múltiples parámetros que resultaría en código difícil de mantener y propenso a errores.

La implementación se estructura mediante la interfaz `PaqueteBuilder` que define el contrato fundamental para la construcción progresiva de paquetes, estableciendo métodos especializados para agregar cada tipo de herramienta ninja en cantidades específicas. El builder proporciona métodos `addKunai()`, `addShuriken()`, `addPapelBomba()`, `addBombaHumo()` y `addBotiquin()` que permiten la composición fluida mediante encadenamiento de llamadas, culminando con el método `build()` que materializa el paquete final con todas las herramientas especificadas.

La clase concreta `PaquetePersonalizadoBuilder` implementa la lógica específica de construcción, manteniendo internamente una lista de herramientas que se va poblando progresivamente mediante cada llamada a los métodos de adición. Esta implementación integra el patrón `Factory` a través de `HerramientaConcreteFactory` para crear instancias individuales de cada herramienta, demostrando la composición efectiva de patrones que maximiza la reutilización de código y mantiene separadas las responsabilidades de construcción y creación.

El `GestorPaquetes` actúa como director del patrón `Builder`, coordinando la construcción de los tres tipos de paquetes predefinidos según las especificaciones exactas requeridas. Para el paquete básico, el gestor dirige la construcción agregando secuencialmente un kunai, un shuriken y un botiquín, mientras que para el paquete avanzado coordina la adición de dos shurikens, tres papeles bomba, dos bombas de humo y dos botiquines. El paquete táctico se construye mediante la dirección coordinada que incorpora tres kunais, dos shurikens, cuatro papeles bomba y dos bombas de humo.

La flexibilidad del patrón `Builder` permite que el sistema maneje tanto la construcción automatizada de paquetes estándar como la creación de configuraciones completamente personalizadas donde el usuario especifica cantidades exactas de cada herramienta. Esta capacidad se materializa en el método

`crearPaquetePersonalizado()` de la clase `Academia`, que acepta parámetros específicos para cada tipo de herramienta y utiliza el builder para construir un paquete único según las especificaciones proporcionadas, validando automáticamente que las cantidades sean no negativas para mantener la integridad de los datos.

2.3. Patrón Factory

¿Por qué se eligió esta implementación?

El patrón Factory fue implementado para encapsular la lógica de creación de elementos complejos del sistema, específicamente herramientas ninja y campos de entrenamiento, proporcionando un mecanismo centralizado que abstrae los detalles de instanciación y permite la extensibilidad futura del sistema sin modificar el código cliente. Esta elección se fundamenta en la necesidad de desacoplar el código que utiliza objetos del código que los crea, facilitando el mantenimiento y la evolución del sistema mediante la centralización de las decisiones de creación en componentes especializados.

La implementación se manifiesta en dos factorías especializadas que manejan diferentes aspectos del dominio ninja. `HerramientaConcreteFactory` implementa la interfaz `HerramientaFactory` para gestionar la creación de las cinco categorías de herramientas ninja, utilizando un mecanismo de selección basado en identificadores string que mapean hacia las clases concretas correspondientes. Esta factoría encapsula la lógica de decisión que determina qué clase instanciar según el tipo solicitado, creando objetos `Kunai`, `Shuriken`, `PapelBomba`, `BombaHumo` o `Botiquin` según la especificación proporcionada.

El sistema de creación de herramientas integra parámetros de cantidad que permiten especificar el peso o cantidad de cada herramienta individual, proporcionando flexibilidad en la configuración de elementos según los requerimientos específicos de cada paquete. La factoría valida automáticamente los parámetros de entrada y maneja casos excepcionales donde se solicitan tipos de herramientas no reconocidos, asegurando la robustez del sistema de creación mediante el manejo apropiado de errores.

Paralelamente, `CampoConcreteFactory` implementa la interfaz `CampoFactory` para gestionar la creación automática de campos de entrenamiento según la suma de niveles de habilidad de los integrantes del grupo. Esta factoría implementa la lógica de negocio que determina automáticamente el campo apropiado, creando instancias de `ValleDelDragon` para grupos con suma menor o igual a siete puntos, `BosqueSombrio` para grupos con suma entre ocho y once puntos, y `MontañaEspiritual` para grupos con suma mayor o igual a doce puntos.

La integración del patrón Factory en el sistema se demuestra en múltiples niveles de la arquitectura, donde `PaquetePersonalizadoBuilder` utiliza `HerramientaConcreteFactory` para crear herramientas individuales durante la construcción de paquetes, mientras que la clase `Grupo` emplea `CampoConcreteFactory` para asignar automáticamente campos de entrenamiento durante la fase de organización. Esta integración muestra la composición efectiva de patrones donde Factory proporciona los elementos fundamentales que Builder organiza en estructuras complejas, resultando en un sistema cohesivo que maximiza la reutilización de código mientras mantiene la flexibilidad para extensiones futuras.

Estado de la Implementación

3.1. Componentes Implementados

La práctica se encuentra **completamente implementada** con todos los componentes requeridos y funcionalidades especificadas en el enunciado original. El desarrollo incluye la correcta implementación de los tres patrones de diseño requeridos (Iterator, Builder y Factory), la integración completa del sistema de gestión de la Academia Ninja con capacidades de manejo automatizado de aspirantes y voluntarios, y un sistema de asignación de recursos que gestiona exitosamente tanto paquetes de herramientas como campos de entrenamiento según las reglas de negocio establecidas.

El sistema incorpora estructuras de datos especializadas que reflejan fielmente los requerimientos del enunciado, utilizando `HashTable` para almacenar información de aspirantes con capacidad de gestionar

al menos diez estudiantes ninja con datos completos incluyendo nombre, edad, clan de procedencia seleccionado entre los cinco clanes especificados, y nivel de habilidad entre uno y tres puntos. La gestión de voluntarios se implementa mediante Array que mantiene información detallada de al menos cinco ninjas instructores con rangos distribuidos entre genin, chunin y jonin, cada uno con niveles de habilidad avanzados entre cuatro y seis puntos.

El subsistema de paquetes de herramientas implementa completamente el patrón Builder mediante la construcción flexible de tres configuraciones predefinidas y paquetes personalizados ilimitados. Los paquetes básicos incluyen exactamente un kunai, un shuriken y un botiquín según especificaciones, mientras que los paquetes avanzados incorporan dos shurikens, tres papeles bomba, dos bombas de humo y dos botiquines para entrenamientos de nivel intermedio. Los paquetes tácticos contienen tres kunais, dos shurikens, cuatro papeles bomba y dos bombas de humo para operaciones especializadas, y el sistema permite la construcción de paquetes completamente personalizados con cantidades arbitrarias de cualquier herramienta disponible.

3.2. Estructura del Proyecto

El proyecto está organizado arquitectónicamente mediante una estructura de paquetes que refleja la separación clara de responsabilidades según los principios de diseño orientado a objetos y facilita el mantenimiento del código. El paquete main contiene la clase principal Main que actúa como punto de entrada del sistema, mientras que el paquete controladores incluye AplicacionControlador y AcademiaFacade que coordinan el funcionamiento integral del sistema mediante el patrón Facade.

La lógica de dominio se estructura mediante el paquete Academia que contiene la clase central Academia responsable de coordinar todas las actividades ninja, junto con subpaquetes especializados que implementan los patrones requeridos. El subpaquete Academia.Listas encapsula la implementación del patrón Iterator con las interfaces Iterator e IterableCollection, las clases contenedoras ListaEstudiantesNinjas y ListaNinjaVoluntarios, y las implementaciones concretas EstudianteNinjaIterator y VoluntarioIterator.

El sistema de construcción de paquetes se organiza dentro del subpaquete Academia.Paquetes, donde la interfaz PaqueteBuilder define el contrato básico, PaquetePersonalizadoBuilder implementa la construcción específica, y GestorPaquetes actúa como director coordinando la construcción de diferentes tipos de paquetes. La gestión de campos se estructura en Academia.Campos con la interfaz CampoFactory, la implementación CampoConcreteFactory, la clase abstracta CampoEntrenamiento, y las tres implementaciones concretas ValleDelDragon, BosqueSombrio y MontañaEspiritual.

Los elementos del mundo ninja se organizan en paquetes especializados que mantienen la cohesión conceptual del dominio. El paquete MundoNinja contiene las clases de entidades fundamentales EstudianteNinja, NinjaVoluntario y Grupo, junto con las enumeraciones ClanesDeProcedencia y Rangos que proporcionan type safety y facilitan la extensibilidad futura. El paquete herramientas implementa el patrón Factory con la interfaz HerramientaFactory, la implementación HerramientaConcreteFactory, la clase abstracta Herramienta, y las cinco implementaciones concretas para cada tipo de herramienta ninja.

Finalmente, el paquete PaquetesHerramientas centraliza las clases de paquetes con la interfaz PaquetesHerramientas, la implementación PaquetePersonalizado, y las tres clases de paquetes predefinidos PaqueteBasico, PaqueteAvanzado y PaqueteTactico, mientras que el paquete ui incluye componentes de interfaz de usuario como MenuPrincipal, GestorInteraccion y EstadoAplicacion, completando una arquitectura modular y bien estructurada que facilita el mantenimiento y la extensibilidad del sistema.

Funcionamiento de la Práctica

El sistema opera mediante un flujo operativo integral que coordina meticulosamente la gestión de aspirantes, voluntarios, formación de grupos, asignación de paquetes de herramientas y campos de entrenamiento, implementando los tres patrones de diseño de manera armoniosa para proporcionar una experiencia de gestión completamente automatizada para la Academia Ninja de la Aldea de las Ciencias. La operación inicia con la inicialización automática de datos que puebla el sistema con al menos diez aspirantes distribuidos entre los cinco clanes disponibles y cinco ninjas voluntarios con rangos diversos,

utilizando el patrón Builder para construir cada entidad con datos consistentes y validados.

Una vez inicializado el sistema, el proceso de formación de grupos utiliza el patrón Iterator para navegar secuencialmente por la colección de ninjas voluntarios almacenados en Array, seleccionando cada instructor disponible como líder potencial de grupo. Para cada voluntario, el sistema determina automáticamente la capacidad máxima de aspirantes según su rango ninja, asignando hasta un aspirante para ninjas genin, hasta dos aspirantes para ninjas chunin, y hasta tres aspirantes para ninjas jonin, reflejando fielmente la jerarquía de experiencia y responsabilidad dentro del sistema ninja.

La asignación de aspirantes a cada grupo se realiza mediante la aplicación del patrón Iterator sobre la HashTable de estudiantes ninja, extrayendo secuencialmente candidatos disponibles hasta completar la capacidad del grupo o agotar los aspirantes sin asignación. Este proceso garantiza que cada aspirante sea asignado exactamente una vez y que no se excedan las capacidades establecidas para cada rango de voluntario, manteniendo la integridad de las reglas de negocio establecidas por la Academia.

Una vez formados los grupos, el sistema procede automáticamente con la asignación de paquetes de herramientas utilizando el patrón Builder coordinado por el GestorPaquetes. El proceso calcula la suma total de niveles de habilidad de todos los integrantes del grupo, incluyendo el líder voluntario y todos los aspirantes asignados, utilizando esta métrica para determinar automáticamente el tipo de paquete apropiado. Los grupos con suma de habilidades relativamente baja reciben paquetes básicos contruidos mediante el builder con un kunai, un shuriken y un botiquín, mientras que grupos con sumas intermedias obtienen paquetes avanzados que incluyen dos shurikens, tres papeles bomba, dos bombas de humo y dos botiquines.

Los grupos con las sumas de habilidades más elevadas reciben paquetes tácticos especializados que incorporan tres kunais, dos shurikens, cuatro papeles bomba y dos bombas de humo, reflejando la capacidad del grupo para manejar herramientas más complejas y peligrosas. El sistema también proporciona capacidad para crear paquetes completamente personalizados mediante la especificación manual de cantidades específicas de cada herramienta, demostrando la flexibilidad del patrón Builder para manejar configuraciones arbitrarias según requerimientos específicos.

La asignación de campos de entrenamiento se ejecuta automáticamente utilizando el patrón Factory implementado en CampoConcreteFactory, que evalúa la suma total de niveles de habilidad del grupo para determinar el ambiente de entrenamiento más apropiado. Los grupos con suma menor o igual a siete puntos son asignados automáticamente al Valle del Dragón, caracterizado por terreno básico ideal para fundamentos ninja y entrenamientos elementales que permiten el desarrollo seguro de habilidades fundamentales.

Los grupos con suma entre ocho y once puntos acceden al Bosque Sombrío, diseñado con obstáculos naturales y desafíos de dificultad intermedia que proporcionan el ambiente apropiado para el desarrollo de técnicas avanzadas sin exceder las capacidades del grupo. Finalmente, los grupos con suma mayor o igual a doce puntos entrenan en la Montaña Espiritual, reservada para entrenamientos de alto nivel que incluyen desafíos extremos apropiados para ninjas con habilidades excepcionales.

El sistema culmina generando un reporte completo que utiliza programación funcional con Streams para procesar y presentar información detallada de todos los grupos formados. Este reporte incluye información de cada líder voluntario con su rango y capacidades, lista completa de aspirantes asignados con sus características individuales, descripción detallada del paquete de herramientas asignado incluyendo cada herramienta individual y peso total calculado, y especificación del campo de entrenamiento asignado con una descripción de sus características y nivel de dificultad. Esta funcionalidad demuestra la integración exitosa de los tres patrones implementados, donde Iterator facilita el acceso a datos, Builder proporciona flexibilidad en la construcción de recursos, y Factory automatiza la creación de elementos especializados, resultando en un sistema robusto y funcionalmente completo que satisface todos los requerimientos especificados para la gestión integral de la Academia Ninja.

Requisitos del Sistema

5.1. Versión de Java

La implementación de esta práctica requiere específicamente **Java Development Kit (JDK) versión 17** o superior para su correcta compilación y ejecución. Esta selección tecnológica se fundamenta en las características avanzadas y la estabilidad que proporciona Java 17 como versión de soporte a largo plazo (LTS), garantizando un entorno de desarrollo robusto y confiable para la implementación de patrones de diseño complejos y la gestión de sistemas orientados a objetos sofisticados que requieren el manejo coordinado de múltiples estructuras de datos y operaciones concurrentes.

Java 17 introduce optimizaciones significativas en el rendimiento de la máquina virtual que benefician directamente la ejecución de los patrones implementados en el sistema de gestión de la Academia Ninja. Las mejoras en la gestión de memoria y el *garbage collection* optimizado facilitan el manejo eficiente de las múltiples instancias de objetos que se crean durante la aplicación del patrón Builder en el sistema de construcción de paquetes, donde cada herramienta adicional genera nuevos objetos que se integran dinámicamente en estructuras complejas mediante encadenamiento de métodos.

Las optimizaciones en el manejo de polimorfismo y *dispatch* de métodos virtuales mejoran significativamente el rendimiento de la implementación del patrón Factory, donde el sistema crea frecuentemente instancias de diferentes tipos de herramientas y campos de entrenamiento según especificaciones dinámicas. Estas mejoras son particularmente relevantes durante la creación intensiva de objetos que ocurre durante la inicialización del sistema y la formación masiva de grupos con sus respectivos recursos asignados.

La versión seleccionada también proporciona mejor soporte para estructuras de datos complejas y manejo de colecciones, aspectos fundamentales para la gestión eficiente de HashTable de aspirantes y Array de voluntarios mediante el patrón Iterator. Java 17 incluye optimizaciones específicas para operaciones de iteración sobre colecciones heterogéneas que mejoran el rendimiento de los algoritmos de formación de grupos y asignación de recursos implementados en el sistema.

Adicionalmente, Java 17 incluye mejoras en el sistema de streams y programación funcional que optimizan las operaciones de procesamiento de datos utilizadas para generar reportes y estadísticas del sistema. Estas características contribuyen a un rendimiento superior durante la generación de resúmenes complejos que involucran múltiples transformaciones de datos y agregaciones estadísticas sobre las estructuras de grupos formados y recursos asignados.

Compilación y Ejecución del Proyecto

6.1. Proceso de Compilación

La compilación del proyecto se realiza mediante el compilador estándar de Java `javac`, aprovechando la estructura modular de paquetes que organiza las clases según sus responsabilidades funcionales específicas dentro del sistema de gestión de la Academia Ninja. El proceso requiere la navegación al directorio `src` del proyecto, desde donde se ejecutan las instrucciones de compilación que procesan recursivamente toda la jerarquía de clases del sistema utilizando un archivo de configuración que especifica la lista completa de archivos fuente.

La arquitectura del proyecto demanda una compilación ordenada que respete las dependencias entre las diferentes clases y paquetes, comenzando por las interfaces fundamentales como `Iterator`, `PaqueteBuilder` y `CampoFactory`, progresando hacia las implementaciones concretas que materializan los patrones de diseño implementados. El sistema de compilación debe procesar las interfaces antes de compilar sus implementaciones concretas, asegurando que todas las referencias se resuelvan correctamente durante el proceso de generación de *bytecode* optimizado.

Para efectuar la compilación completa del sistema, se debe posicionar en el directorio `src` del proyecto y ejecutar la instrucción de compilación que utiliza el archivo `files.txt` conteniendo la lista ordenada

de todos los archivos fuente Java distribuidos en la estructura de paquetes. El comando específico para compilar todos los archivos del sistema es:

```
javac -cp . @files.txt
```

Este proceso se realiza mediante la aplicación sistemática del compilador `javac` utilizando el parámetro `@files.txt` que instruye al compilador para leer la lista de archivos desde el archivo especificado, respetando la jerarquía de dependencias establecida durante la fase de desarrollo. El parámetro `-cp .` establece el classpath para incluir el directorio actual en la búsqueda de clases, permitiendo la resolución correcta de todas las dependencias entre paquetes.

La compilación exitosa genera archivos de bytecode con extensión `.class` que mantienen la misma estructura jerárquica de directorios que el código fuente, permitiendo que la máquina virtual de Java localice y cargue correctamente todas las clases durante la ejecución posterior. Este proceso produce archivos optimizados que aprovechan las características específicas de Java 17 para proporcionar rendimiento óptimo durante la simulación completa del sistema de gestión de la Academia Ninja.

6.2. Ejecución del Sistema

La ejecución del sistema se inicia mediante el comando estándar de Java `java`, especificando la clase principal `main.Main` que actúa como punto de entrada de toda la aplicación y coordina la inicialización completa del sistema de gestión de actividades ninja. La ejecución debe realizarse desde el directorio `src` del proyecto, donde se encuentran disponibles los archivos bytecode compilados organizados según la estructura de paquetes establecida durante la fase de desarrollo.

El comando de ejecución específico para iniciar el sistema completo utiliza el siguiente formato, ejecutado desde el directorio `src`:

```
java -cp .:main main.Main
```

Este comando incorpora el parámetro `-cp .:main` que establece el classpath para incluir tanto el directorio actual como el subdirectorio `main` en la búsqueda de clases, permitiendo que la máquina virtual localice apropiadamente todas las clases distribuidas en los diferentes paquetes del sistema. La instrucción activa la máquina virtual de Java, carga todas las clases necesarias según las dependencias del sistema, e inicia la ejecución secuencial del flujo operativo completo de la Academia Ninja.

Durante la ejecución, el sistema presenta información detallada en la consola mostrando cada etapa del proceso operativo, incluyendo la inicialización automática de aspirantes y voluntarios, la formación progresiva de grupos según las reglas de negocio, la aplicación de los patrones de diseño durante la asignación de recursos, y la generación de reportes finales con estadísticas completas. La salida en tiempo real permite supervisar el funcionamiento correcto de todos los patrones implementados, evidenciando la navegación secuencial mediante el patrón `Iterator`, la construcción progresiva mediante el patrón `Builder`, y la creación automática mediante el patrón `Factory`.

El flujo completo de operación del sistema permite experimentar interactivamente con todos los aspectos implementados, desde la gestión automatizada de datos hasta la asignación inteligente de recursos mediante los patrones de diseño, proporcionando una demostración integral de la funcionalidad especificada para la Academia Ninja de la Aldea de las Ciencias completamente automatizada. El sistema culmina presentando un resumen detallado que demuestra la integración exitosa de todos los componentes y la aplicación correcta de los tres patrones de diseño requeridos en un contexto práctico y funcional.